# Software Design Document

# Software Specification Document for Example System

## 1. Purpose

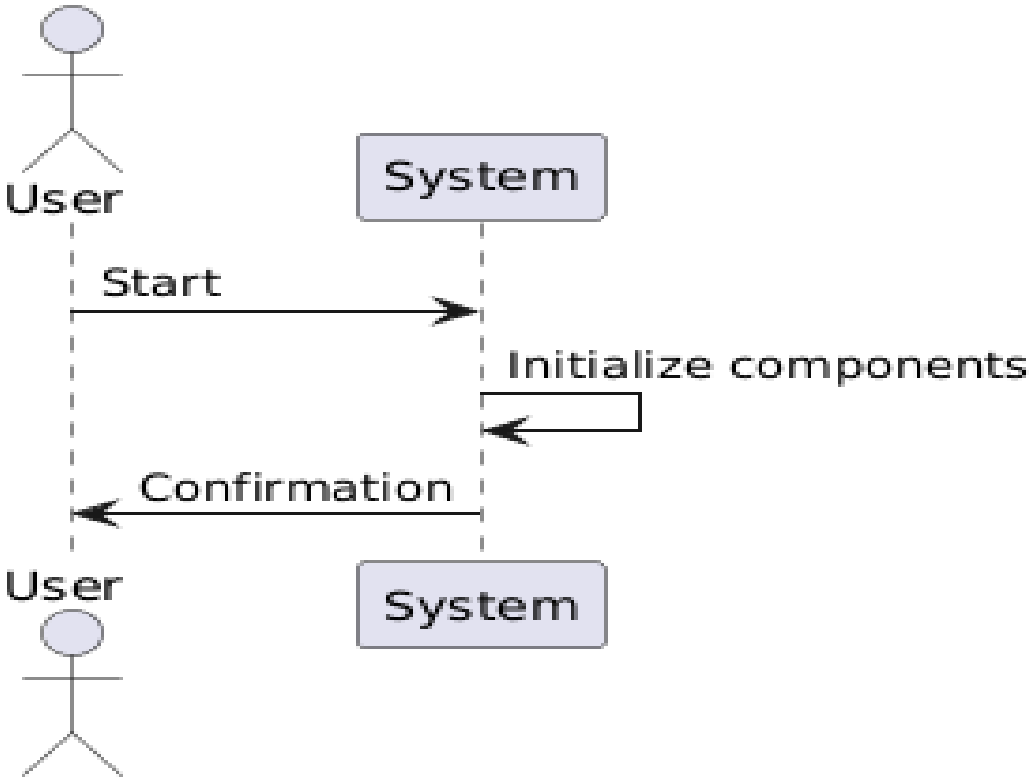
The purpose of this document is to provide a detailed plan for building a simple example software system. This system is designed as a means to verify the functionality of code. The document outlines the architecture, use cases, sequence diagrams, design decisions, and testing strategy for the system.

## 2. Sequence Diagrams

#### Use Case 1: System Initialization

- **Description**: The system starts and initializes necessary components.

```plaintext
```

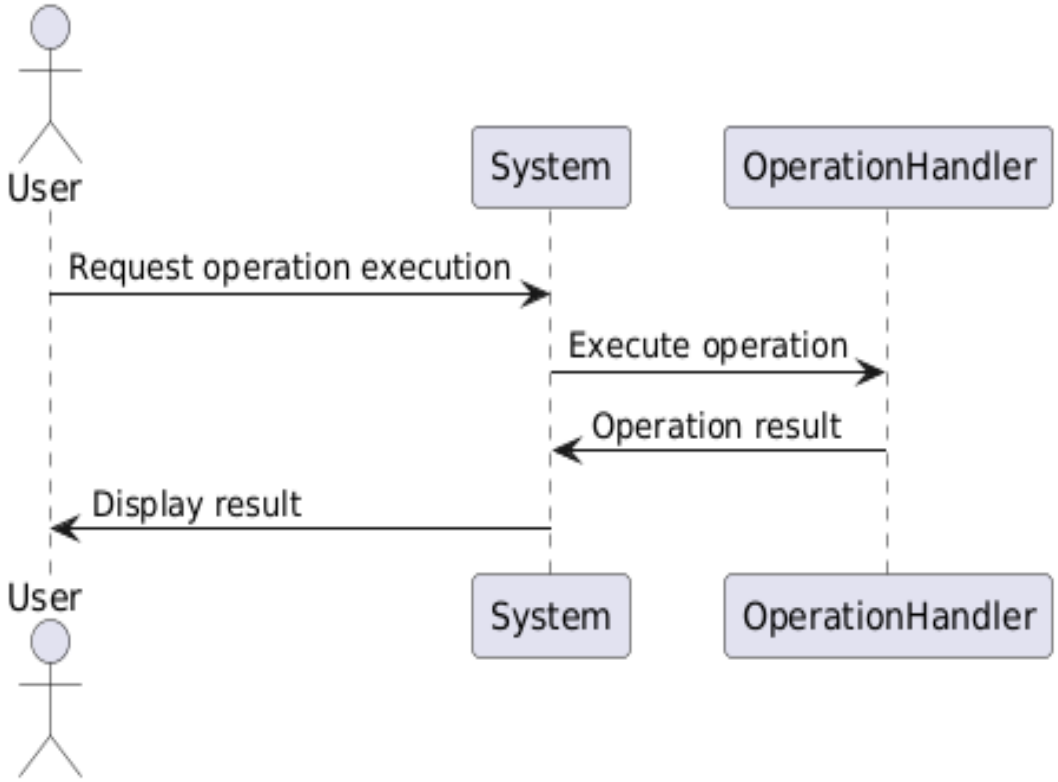



```
```

**Use Case 2: Operation Execution**

- **Description**: The system executes a predefined operation upon user request.

```plaintext
```




```
```

## 3. Major Design Decisions

- **Cohesion and Coupling**:

- The system will be designed to maximize cohesion within components by grouping related functionalities together. Coupling will be minimized by ensuring components interact through well-defined interfaces.

- **Non-Functional Requirements**:

- *Reliability*: The system should maintain consistent operation even in edge cases.

- *Performance*: The system should execute operations with minimal latency.

## 4. Architecture

The architecture of the system employs the Layered Pattern to separate concerns and improve maintainability. The system is divided into three primary layers: Presentation, Business Logic, and Data Access. Communication between layers is strictly organized to ensure smooth interactions.

**Paragraph 1: Architectural Patterns**

The chosen architectural pattern is the Layered Pattern. This pattern allows the system to organize code logically and separate concerns, which improves maintainability and scalability. By dividing functionalities across multiple layers, changes in one layer (e.g., data access) do not directly affect the others, promoting modularization.

**Paragraph 2: Initial Decomposition**

The system is decomposed into the following modules: User Interface, Operation Handler, and Data Manager. Each module performs specific roles: the User Interface manages interactions with the user, the Operation Handler executes user requests, and the Data Manager handles data storage and retrieval.

```plaintext


```

```

**Component Functions and Interfaces**

- **User Interface**:

- *DisplayPrompt()*: Displays user prompts.

- *ShowResult(result)*: Shows the result to the user.

- **Operation Handler**:

- *ExecuteOperation()*: Handles execution of the requested operation.

- **Data Manager**:

- *RetrieveData()*: Retrieves necessary data for operations.

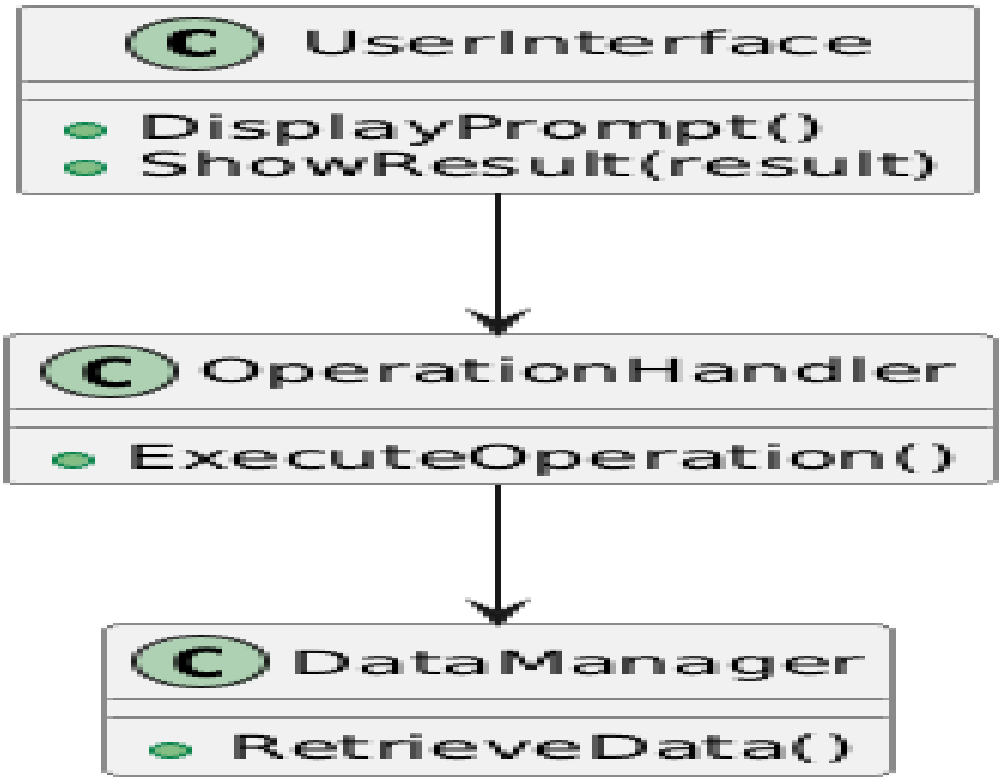
## 5. Use of Design Patterns

- **MVC (Model-View-Controller)**: This pattern is used to separate internal representations from the way information is presented to and accepted from the user. It increases modularity and allows multiple views to represent the same data.

- **Singleton Pattern**: Employed in the Data Manager to ensure that a single instance of data access logic is used throughout the system, which conserves resources and ensures consistency.

## 6. Detailed Class Diagrams

```plaintext




```

## 7. Test Driven Development

**Test Case 1**

1. **Test ID**: TC_01

2. **Category**: System Initialization

3. **Initial Condition**: System is not running.

4. **Procedure**:

- Start the system.

- Check system log for initialization message.

5. **Expected Outcome**: System initializes and logs a confirmation message.

**Test Case 2**
```

1. **Test ID**: TC_02

2. **Category**: Operation Execution

3. **Initial Condition**: System is running and ready.

4. **Procedure**:

- Send an operation execution request.

- Verify the correctness of the response.

5. **Expected Outcome**: Correct operation results are displayed to the user.